

TAP AGAINST ME

Competitive Dynamic Day/Night and Gravity Presentation Bible

Document version: 1.0

Feature scope: Competitive matches, ranked matches, head-to-head matches, private competitive matches, bot-supported competitive matches, and competitive spectating

Primary platform: Android-first, portrait orientation

Feature owner: Don't Watch Media

Project: Tap Against Me

Cycle duration: 120 seconds

Typical competitive survival time: Approximately 70 seconds

Feature status: Approved design direction; implementation details subject to playtesting and performance validation

1. Feature Summary

Tap Against Me's competitive gameplay becomes progressively more difficult as gravity increases. Previously, the visual presentation connected higher gravity with a darker environment. As gravity became stronger, the world gradually appeared to turn toward night.

That relationship is no longer the desired design.

Darkness does not intuitively communicate gravity. It can also cause players to assume that nighttime has a gameplay effect, that gravity is controlled by the sky, or that night itself represents a punishment.

The game must instead treat these as two independent systems:

Gravity communicates gameplay difficulty through physics, motion, impact, and pressure.

The day/night cycle communicates time, atmosphere, visual progression, and the feeling that the world continues moving during a match.

Competitive matches will use an accelerated, continuous day/night cycle lasting **two minutes from one full cycle to the next**. The sun and moon will visibly travel through the sky. Lighting, sky colors, stars, shadows, environmental ambience, and cosmetic biome reactions will transition naturally as the match progresses.

The day/night cycle must remain competitively neutral. It cannot alter physics, player visibility, collision, item behavior, spawn probabilities, or the relative difficulty experienced by either competitor.

2. Core Design Intent

The system should accomplish five major goals.

2.1 Make the Competitive World Feel Alive

Competitive matches should not feel as though they take place against a static wallpaper. Even though the core action is focused and mechanically simple, the world around the players should continue changing.

During a match, players may witness:

- Sunrise.
- A brightening sky.
- The sun moving overhead.
- Afternoon light.
- Sunset.
- Twilight.
- Stars appearing.
- The moon rising.
- Nighttime environmental activity.
- Predawn light returning.

The match environment should feel like a small world moving through time rather than an unmoving arena.

2.2 Give Long Survival Runs a Visual Sense of Accomplishment

A player who survives for only a few seconds will experience a limited portion of the environmental cycle.

A player surviving for approximately 70 seconds will witness a substantial transition.

A player surviving for two minutes will have remained airborne through an entire accelerated day.

The changing environment therefore becomes a subtle visual indicator of endurance without replacing the actual timer or score.

2.3 Increase Match-to-Match Variety

Competitive matches are generally short. If every match always begins at exactly the same time of day, most players will repeatedly see the same portion of the cycle.

The system should eventually support synchronized, seed-controlled starting phases so one match may begin at sunrise, another during daylight, another near sunset, and another during the night.

Both competitors must always receive the same environmental phase.

2.4 Preserve Competitive Readability

The environment may become darker, warmer, cooler, brighter, or more atmospheric, but the player must never lose because a cosmetic transition obscured something important.

Characters, hazards, fruit, Hero Tokens, effects, and critical gameplay information must remain readable during every phase.

2.5 Separate Mechanical Difficulty from Environmental Atmosphere

Gravity must remain a gameplay system.

The day/night cycle must remain an environmental presentation system.

The systems may respond to the same match timer, but neither should calculate or control the other.

3. Canonical Design Rules

The following requirements are considered locked unless the feature owner explicitly revises them.

3.1 Locked Requirements

1. A complete day/night cycle lasts **120 seconds**.
2. The cycle transitions continuously. It must not abruptly switch between four static backgrounds.
3. The sun visibly travels through the sky.
4. The moon visibly travels through the sky.
5. The sun and moon are environmental visuals and have no collision.
6. Gravity does not directly darken the sky.
7. Time of day does not directly change gravity.
8. Nighttime must not make the competitive game materially harder to see.
9. Both players in a synchronized match experience the same time of day.
10. Day/night synchronization must be derived from authoritative match data rather than each client independently choosing a start time.

11. The environment remains cosmetic unless a future mode explicitly declares environmental gameplay modifiers.
12. Purchases, skins, effects, or premium content cannot alter the competitive cycle, visibility rules, gravity, or environmental advantages.
13. The user interface does not darken with the world.
14. Gameplay objects retain adequate contrast during every phase.
15. Environmental visual changes may communicate gravity, but sky darkness cannot be used as the primary indicator of gravity strength.

3.2 Recommended Requirements

The following are strongly recommended but may be adjusted through playtesting:

- Competitive matches use a server-selected or seed-selected starting phase.
 - Starting phases are chosen from curated phase anchors rather than completely uncontrolled random values.
 - The sun and moon remain approximately opposite each other within the full cycle.
 - Stars fade in gradually.
 - Environmental ambience crossfades rather than switches.
 - Shadows change gradually with celestial movement.
 - Biome-specific reactions remain cosmetic.
 - The editor uses neutral lighting by default, with an optional gameplay-lighting preview.
 - Strong lighting changes are reduced automatically on lower-performance devices.
-

4. Feature Scope

4.1 Included Modes

This specification applies to:

- Ranked player-versus-player matches.
- Public competitive matchmaking.
- Head-to-head private matches.
- Bot-backfilled competitive matches.
- Competitive matches against simulated or ghost opponents.
- Solo competitive survival runs where standard competitive rules are used.
- Competitive replays.
- Competitive spectating.

4.2 Infinity Taps

Infinity Taps is not governed by the exact pacing requirements in this document.

Infinity Taps is relaxed, exploratory, open-ended, and biome-driven. It may eventually use:

- Longer day/night cycles.
- Biome-specific cycle lengths.
- Weather systems.
- Seasonal lighting.
- Persistent local time.
- Special nighttime discoveries.
- Nocturnal animals.
- Hidden interactions.
- Rare celestial events.

Those systems should be designed separately because Infinity Taps serves a different player experience.

Shared shaders, sky resources, celestial assets, and lighting controllers may be reused, but Infinity Taps should not be forced into the 120-second competitive cycle.

5. Player Experience

The player should perceive the system as a natural part of the world rather than as a visible timer with four background states.

A player should be able to notice that time is moving without needing to consciously track the sun.

The ideal experience is:

1. The player begins the match.
2. The environment is already in a clearly identifiable time-of-day state.
3. The sun or moon continues moving while gameplay occurs.
4. The sky and ambient lighting transition subtly.
5. Environmental details respond to the changing light.
6. Gravity increases according to the competitive difficulty curve.
7. Gravity feels increasingly heavy through movement and feedback.
8. The environment progresses independently of that difficulty.
9. The match ends without either system appearing to cause the other.

The cycle must enhance the match without competing for attention.

6. Two-Minute Cycle Structure

The cycle is continuous, but development should divide it into conceptual phases for authoring colors, lighting, ambience, and reactions.

The following phase timing is the recommended baseline.

Match-cycle time	Normalized phase	Environmental state
0–12 seconds	0.00–0.10	Sunrise
12–42 seconds	0.10–0.35	Morning and daylight
42–60 seconds	0.35–0.50	Afternoon and sunset
60–72 seconds	0.50–0.60	Twilight
72–108 seconds	0.60–0.90	Night
108–120 seconds	0.90–1.00	Predawn and returning sunrise

At 120 seconds, the cycle wraps seamlessly back to zero.

These are authoring ranges, not hard scene changes. Every visual property should interpolate across the phase boundaries.

6.1 Sunrise: 0–12 Seconds

Desired presentation:

- A soft glow develops near the horizon.
- The sky transitions from dark blue or violet toward warm orange.
- Stars fade away.
- The sun becomes visible at the horizon.
- Ambient brightness begins increasing.
- Nighttime environmental details fade out.
- Morning details may briefly appear, such as mist or distant birds.

Sunrise should feel quick enough to fit the accelerated cycle without appearing like an abrupt light switch.

6.2 Morning and Daylight: 12–42 Seconds

Desired presentation:

- The sun rises higher.
- The sky settles into the biome's daytime palette.
- Ambient lighting reaches its clearest state.
- Shadows become shorter as the sun approaches its highest point.

- Day-active creatures and cosmetic reactions become more likely.
- Water and reflective surfaces may show sunlight highlights.

This should be the cleanest and most visually neutral competitive phase.

6.3 Afternoon and Sunset: 42–60 Seconds

Desired presentation:

- The sun moves toward the opposite horizon.
- Light becomes warmer.
- The sky gradually introduces orange, rose, red, and purple values appropriate to the biome.
- Shadows lengthen.
- Sunlight reflection changes direction.
- Day-active environmental details begin settling.

Sunset must not overwhelm orange, red, yellow, or gold gameplay pickups.

Golden Hero Tokens must remain distinct even against a warm sunset.

6.4 Twilight: 60–72 Seconds

Desired presentation:

- The sun disappears below the horizon.
- The moon begins becoming visible.
- The sky transitions from warm sunset colors into deeper blue and violet.
- Stars begin appearing gradually.
- Artificial lights, fireflies, glowing plants, or windows may begin illuminating.
- The gameplay layer receives compensating rim lighting or contrast support.

Twilight should be one of the richest visual phases, but it must not become noisy.

6.5 Night: 72–108 Seconds

Desired presentation:

- The moon travels across the sky.
- Stars are fully visible.
- The biome uses its nighttime palette.
- Nocturnal ambience becomes active.
- Background details may become darker.
- Characters and interactive objects remain readable.
- Night-active animals or environmental animations may appear.
- Water may reflect moonlight.

Night must feel atmospheric rather than visually punishing.

6.6 Predawn: 108–120 Seconds

Desired presentation:

- The moon moves toward the horizon.
- Stars begin fading.
- A subtle cool or pale glow develops near the sunrise horizon.
- Nighttime ambience decreases.
- The world prepares to return to sunrise.
- The loop reconnects seamlessly to the first phase.

There must be no visible flash, palette reset, sprite teleportation, or lighting discontinuity when the cycle wraps.

7. Starting Phase Rules

7.1 Initial Implementation

During the first implementation phase, all matches may begin at a fixed phase while the cycle is validated.

Recommended first test start:

- Normalized cycle phase:
- Time of day: Sunrise
- Cycle duration: 120 seconds

A fixed start makes visual bugs, timing errors, and synchronization problems easier to reproduce.

7.2 Final Competitive Behavior

After synchronization and readability are stable, competitive matches should support varied starting phases.

Recommended starting anchors:

- Sunrise:
- Midday:
- Sunset:
- Early night:
- Midnight:
- Predawn:

The match server or authoritative match creator selects the starting anchor using the match seed.

The environment should not select a new phase separately on each device.

7.3 Why Curated Anchors Are Preferred

A completely random normalized value could repeatedly place players in visually awkward transition moments.

Curated anchors provide:

- Recognizable starting scenes.
- Reliable visual testing.
- Better art direction.
- Balanced exposure to each phase.
- Easier screenshots and promotional capture.
- More predictable accessibility validation.

A minor deterministic offset may be added around each anchor later to prevent exact repetition.

7.4 Ranked Fairness

Starting phase is cosmetic.

It must not influence:

- Gravity progression.
- Tap force.
- Character movement.
- Collision.
- Fruit spawn timing.
- Fruit spawn position.
- Hero Token spawn timing.
- Bot behavior.
- Matchmaking rating.
- Scoring.
- Input handling.
- Network latency compensation.

8. Celestial Movement

8.1 Sun Path

The sun should travel along an arc from one horizon to the opposite horizon.

In portrait orientation, the arc should be designed to avoid placing the sun directly behind the densest gameplay region for extended periods.

A basic implementation can use:

```
x = lerp(left_horizon_x, right_horizon_x, sun_progress)
y = horizon_y - arc_height * sin(PI * sun_progress)
```

Where:

- `sun_progress` ranges from `0.0` at sunrise to `1.0` at sunset.
- `arc_height` is biome-configurable.
- The sun is hidden or moved below the visible horizon outside its active range.

The final path may use an animation curve rather than a raw sine function.

8.2 Moon Path

The moon follows a similar arc during the night portion.

The moon should generally appear opposite the sun in the complete cycle. However, strict astronomical simulation is unnecessary.

The visual priority is:

- Clear transition.
- Recognizable movement.
- Attractive composition.
- Seamless looping.
- Low distraction.

8.3 Layering

The recommended draw order is:

1. Base sky gradient.
2. Distant stars.
3. Sun or moon.
4. Distant clouds.
5. Far background.
6. Mid-background.
7. Near background.
8. Competitive gameplay layer.
9. Foreground effects.
10. Interface.

Cloud placement must not repeatedly obscure celestial bodies in a visually distracting way.

8.4 Celestial Interaction

The sun and moon:

- Have no collision.
- Cannot be tapped in standard competitive mode.
- Do not create pickups.
- Do not alter physics.
- Do not affect item rarity.
- Do not blind the player.
- Do not create lens flares over gameplay objects.
- Do not produce flashes.

Infinity Taps may eventually allow sun or moon interactions, but that is outside this specification.

9. Sky and Lighting System

9.1 Continuous Interpolation

Sky presentation should be controlled by curves or gradients sampled from normalized cycle time.

Properties may include:

- Top sky color.
- Horizon color.
- Lower sky color.
- Ambient light intensity.
- Ambient light temperature.
- Background saturation.
- Background contrast.
- Star opacity.
- Cloud brightness.
- Moon opacity.
- Sun opacity.
- Artificial light intensity.
- Character rim-light intensity.
- Shadow opacity.
- Fog or mist opacity.

The system must avoid large linear jumps caused by manually switching states.

9.2 Recommended Sky Profile Resource

Each biome should eventually use a reusable environment profile containing:

```
DayNightProfile
- cycle_duration
- sky_top_gradient
- sky_horizon_gradient
- ambient_intensity_curve
- ambient_temperature_gradient
- star_opacity_curve
- sun_opacity_curve
- moon_opacity_curve
- shadow_strength_curve
- rim_light_curve
- background_saturation_curve
- artificial_light_curve
- ambience_mix_curve
- celestial_arc_settings
- performance_fallback_settings
```

The cycle controller owns time. The profile owns visual interpretation.

9.3 Gameplay Layer Protection

Background lighting may change dramatically.

The gameplay layer must use protected readability rules.

Recommended protections include:

- A minimum character brightness.
- A minimum contrast ratio against the active background.
- A subtle character outline.
- Nighttime rim lighting.
- Fixed or limited-range pickup emissive values.
- A background dimming mask behind critical objects where needed.
- Reduced saturation directly behind gameplay objects.
- Controlled particle brightness.

The system should not simply increase the brightness of everything at night. That would flatten the visual hierarchy.

9.4 User Interface

The user interface remains visually stable.

The following elements should not inherit the world lighting:

- Match timer.
- Player names.
- Health or status indicators.
- Gravity meter.
- Item indicators.
- Connection warnings.
- Ranking information.
- Pause controls where applicable.
- Accessibility prompts.
- End-of-match interface.

Minor cosmetic theme changes may be considered later, but the interface must remain immediately readable.

10. Gravity System Separation

10.1 Gameplay Gravity

Competitive gravity should continue using its authoritative difficulty rules.

A conceptual effective-gravity calculation may be:

```
effective_gravity =  
    base_gravity  
    + time_based_gravity_ramp  
    + active_debuff_gravity  
    + mode_specific_gravity_modifier
```

Purple Fruit gravity effects should be included as an explicit gameplay modifier.

The day/night controller must not appear in this calculation.

10.2 Forbidden Relationship

The following pattern must not remain in the final implementation:

```
sky_darkness = gravity_strength
```

or:

```
night_progress = effective_gravity / maximum_gravity
```

This couples atmosphere to difficulty and makes future balancing changes alter the art presentation unintentionally.

10.3 Correct Relationship

The systems may use the same elapsed match clock but calculate independently:

```
gravity_strength = gravity_profile.evaluate(match_elapsed_time)

day_night_phase =
    (match_start_phase + match_elapsed_time / cycle_duration) % 1.0
```

This allows gravity curves to be rebalanced without changing the environment.

It also allows the environment to begin at different phases without altering gameplay.

11. Visual Communication of Gravity

Removing darkness as a gravity indicator does not mean gravity should have no visual feedback.

Gravity should be communicated through physical behavior.

11.1 Increasing Gravity Feedback

As effective gravity increases, the game may gradually introduce:

- Faster downward acceleration.
- Stronger downward particle drift.
- Heavier character squash during sharp downward movement.
- More forceful tap recovery animations.
- Slightly stronger camera impulse at major gravity thresholds.
- Faster falling leaves, rain, snow, dust, or debris.
- Greater trailing distortion below the player.
- More pronounced wind streaks during descent.
- Heavier environmental sway.

- Stronger ground-impact anticipation.
- A subtle pressure vignette at extreme gravity.
- More intense gravity-meter animation.
- Deeper or denser gravity-related audio layers.

These effects must remain subtle enough to avoid obscuring inputs or character position.

11.2 Low Gravity Feedback

Modes or effects that temporarily reduce gravity may use:

- Slower particle descent.
- Longer character hang time.
- More floating accessories.
- Gentle upward dust or pollen.
- Extended animation arcs.
- Softer impact sounds.
- Reduced squash.
- Wider trailing motion.

11.3 Purple Fruit Debuff

Purple Fruit should have a distinct gravity-debuff presentation separate from normal time-based gravity growth.

Potential feedback:

- Brief purple pulse around the affected player.
- Downward purple streak.
- Temporary gravity icon.
- A short low-frequency impact sound.
- A visible but non-obstructive debuff timer.
- Increased downward-motion feedback.

The sky must not turn purple or darken globally because one player collected a Purple Fruit.

In head-to-head play, one player's debuff cannot alter the other player's environment.

12. Biome Integration

The core day/night controller should be biome-independent.

Each biome interprets the shared normalized phase through its own profile.

12.1 Shared Rules

Every competitive biome must provide:

- Day palette.
- Sunset palette.
- Night palette.
- Sunrise palette.
- Sun presentation.
- Moon presentation.
- Star behavior.
- Protected gameplay contrast.
- Ambient sound transitions.
- Performance fallback.

12.2 Biome-Specific Reactions

Cosmetic reactions may include:

- Flowers opening during daylight.
- Flowers closing at night.
- Fireflies appearing after sunset.
- Rabbits becoming less active at night.
- Owls appearing in trees.
- Bats crossing the distant sky.
- Moths gathering around lights.
- Windows or lanterns illuminating.
- Glowing mushrooms becoming more visible.
- Water reflecting the sun or moon.
- Morning mist appearing near sunrise.
- Crickets becoming audible at night.
- Birds becoming audible during morning.
- Leaves changing highlight direction.
- Snow reflecting moonlight.
- Desert temperatures being implied through color.
- City signs or streetlights turning on.

All biome reactions remain cosmetic in competitive play.

12.3 Competitive Restraint

The competitive version of a biome should be less visually busy than its Infinity Taps version.

Competitive priorities are:

1. Player position.

2. Opponent position.
3. Hazards.
4. Pickups.
5. Match information.
6. Environmental spectacle.

The environment should reward peripheral attention without demanding primary attention.

13. Environmental Audio

13.1 Music

Competitive music should continue independently of the day/night system unless a specific adaptive soundtrack is authored.

The day/night cycle should not restart the music every 120 seconds.

Recommended approaches:

- Keep the competitive track uninterrupted.
- Add subtle environmental layers beneath it.
- Crossfade those layers according to cycle phase.
- Preserve musical timing and intensity independently from environmental time.

13.2 Ambient Layers

Possible ambient layers include:

- Morning birds.
- Daytime wind.
- Evening insects.
- Nighttime crickets.
- Distant owls.
- Soft nighttime air.
- Dawn ambience.
- Biome-specific environmental loops.

Crossfades should be gradual.

No ambience should mask:

- Tap feedback.
- Item collection.
- Debuff notifications.
- Match-start cues.

- Victory or defeat cues.
- Connection warnings.

13.3 Gravity Audio

Gravity should have its own audio presentation.

Possible gravity feedback:

- A progressively stronger low-frequency layer.
- Denser downward movement sounds.
- Increased impact weight.
- Threshold cues when gravity reaches major stages.
- A distinct Purple Fruit debuff sound.

Gravity audio should not be tied to nighttime ambience.

14. Networking and Synchronization

14.1 Authoritative Match Data

The server or authoritative match host should provide:

```
match_id
environment_seed
day_night_profile_id
cycle_duration
starting_phase
authoritative_match_start_time
biome_id
environment_variant_id
```

For the standard competitive mode:

```
cycle_duration = 120.0 seconds
```

14.2 Client Phase Calculation

Clients should calculate the current phase from authoritative elapsed match time:

```
elapsed =
    synchronized_server_time
    - authoritative_match_start_time

phase =
    (starting_phase + elapsed / cycle_duration) % 1.0
```

The server does not need to transmit sky colors every frame.

14.3 Drift Correction

Clients should periodically compare local estimated match time with authoritative time.

Small discrepancies should be corrected gradually.

Large discrepancies caused by reconnecting or resuming should snap or rapidly blend to the correct phase.

Recommended behavior:

- Less than 50 milliseconds drift: Ignore.
- 50–250 milliseconds: Smooth correction.
- Greater than 250 milliseconds: Accelerated correction.
- Reconnection: Restore authoritative phase immediately with a short visual blend where possible.

The exact thresholds should be validated against the networking implementation.

14.4 Determinism

The following should be deterministic from the environment seed where appropriate:

- Starting phase.
- Selected environment variant.
- Decorative cloud pattern.
- Cosmetic background animal events.
- Star pattern variation.
- Optional distant environmental events.

Deterministic cosmetic events ensure both players and replays can reproduce the same presentation.

14.5 Replays and Spectators

Replays should store or reconstruct:

- Environment seed.
- Starting phase.

- Cycle duration.
- Biome profile.
- Match start time relative to replay zero.

Spectators should calculate the environment from the same authoritative match timeline.

14.6 Bots

Bots experience the same gameplay gravity as players.

The visual environment does not need to be simulated for bot decision-making because it is competitively cosmetic.

Bots must not gain targeting or timing advantages from nighttime or reduced visual clutter.

15. Offline and Pause Behavior

15.1 Online Competitive Matches

Online matches do not pause the authoritative environment timeline.

Opening an overlay or losing application focus does not pause:

- Gravity.
- Match time.
- Day/night progression.
- Opponent movement.
- Spawn timing.

15.2 Offline Competitive Practice

Offline practice may pause the complete simulation.

When paused:

- Gravity progression freezes.
- Character physics freeze.
- Spawn timers freeze.
- Day/night progression freezes.
- Environmental animations may freeze or move at a reduced menu rate.

The implementation should use the same simulation clock for all match-dependent systems to avoid desynchronization.

15.3 Pre-Match Countdown

The day/night phase should generally remain fixed during the short pre-match countdown.

When the match begins, the environment timeline starts with gameplay.

This prevents the match from beginning several seconds beyond its assigned starting phase because of network countdown timing.

16. Proposed Technical Architecture

The exact node names may be adapted to the project, but responsibilities should remain separated.

16.1 MatchEnvironmentController

Responsibilities:

- Receives authoritative environment configuration.
- Tracks normalized cycle phase.
- Emits phase updates.
- Handles synchronization and drift correction.
- Loads the correct biome profile.
- Coordinates environment subsystems.
- Does not calculate gravity.

Suggested signals:

```
phase_changed(normalized_phase)
environment_initialized(profile_id, seed)
major_phase_entered(phase_name)
environment_resynchronized(correction_amount)
```

16.2 DayNightProfile Resource

Responsibilities:

- Stores biome-specific gradients.
- Stores lighting curves.
- Stores sun and moon paths.
- Stores ambience curves.
- Stores performance fallbacks.
- Contains no player-specific state.

16.3 CelestialController

Responsibilities:

- Positions sun.
- Positions moon.
- Controls celestial opacity.
- Controls optional glow.
- Keeps celestial objects within safe visual regions.
- Handles horizon hiding.

16.4 SkyRenderer

Responsibilities:

- Draws sky gradient.
- Blends phase colors.
- Controls stars.
- Controls cloud brightness.
- Applies background-only lighting effects.
- Exposes lower-cost fallback modes.

16.5 GameplayReadabilityController

Responsibilities:

- Maintains player contrast.
- Applies nighttime rim lighting.
- Protects pickup readability.
- Controls minimum brightness.
- Responds to accessibility settings.
- Does not alter collisions or gameplay.

16.6 GravityPresentationController

Responsibilities:

- Reads effective gameplay gravity.
- Produces gravity-specific particles and animation feedback.
- Controls pressure effects.
- Responds to Purple Fruit debuffs.
- Does not control sky phase.

16.7 BiomeReactionController

Responsibilities:

- Activates cosmetic day creatures.
- Activates cosmetic night creatures.
- Controls lights, plants, water reflections, and atmospheric effects.
- Uses deterministic seeded timing where synchronized presentation is desired.

16.8 EnvironmentalAudioController

Responsibilities:

- Crossfades ambient layers.
- Keeps ambience below gameplay audio.
- Responds to biome and cycle phase.
- Does not restart competitive music.

17. Godot-Oriented Implementation Notes

The feature should remain modular even if the current project uses custom rendering or shaders.

A possible scene structure is:

```
CompetitiveMatch
├─ MatchSimulation
├─ MatchEnvironmentController
│   └─ SkyRenderer
│   └─ CelestialController
│       └─ Sun
│       └─ Moon
│   └─ Stars
│   └─ CloudLayer
│   └─ BiomeReactionController
│   └─ EnvironmentalAudioController
│   └─ GameplayReadabilityController
├─ GravityPresentationController
├─ PlayerLayer
├─ PickupLayer
├─ EffectsLayer
└─ InterfaceLayer
```

Recommended update pattern:

```
func _process(_delta: float) -> void:
    var elapsed := match_clock.get_authoritative_elapsed_time()
    var phase := fposmod(
        starting_phase + elapsed / cycle_duration,
        1.0
    )

    apply_environment_phase(phase)
```

Environment properties should be sampled from resources:

```
func apply_environment_phase(phase: float) -> void:
    sky_renderer.set_top_color(profile.sky_top_gradient.sample(phase))
    sky_renderer.set_horizon_color(profile.sky_horizon_gradient.sample(phase))
    sky_renderer.set_star_opacity(profile.star_opacity_curve.sample(phase))
    celestial_controller.set_phase(phase)
    readability_controller.set_phase(phase)
    audio_controller.set_phase(phase)
```

Gravity remains separate:

```
func update_gravity_feedback() -> void:
    var effective_gravity := gravity_system.get_effective_gravity()
    gravity_presentation.set_strength(effective_gravity)
```

There should be no call resembling:

```
environment_controller.set_darkness(effective_gravity)
```

17.1 Editor Lighting

The editor should default to fixed neutral lighting so developers can reliably inspect:

- Sprite colors.
- Collision boundaries.
- Item readability.
- Animation.
- Scene composition.
- Character variants.

An optional **Preview Game Lighting** control may run the day/night cycle in the editor.

Preview mode must not permanently modify authored sprite colors or materials.

18. Performance Requirements

Tap Against Me is Android-first and must perform reliably on a wide range of phones.

18.1 Performance Priorities

The environment should prefer:

- One efficient sky shader or gradient renderer.
- Simple sun and moon sprites.
- Pooled particles.
- Shared material resources.
- Low-frequency environmental updates where possible.
- Curves sampled efficiently.
- Minimal full-screen post-processing.
- Limited transparency overdraw.
- Preloaded biome profiles.
- No per-frame allocation.

18.2 Update Frequency

Not every system needs to update every rendered frame.

Recommended starting points:

System	Suggested update behavior
Sky gradient	Every frame or shader-driven
Sun and moon position	Every frame
Stars	Shader-driven or 10–20 Hz
Ambient audio mix	5–10 Hz
Background creature scheduling	Event-driven
Artificial light intensity	10–20 Hz
Readability correction	Every frame or material-driven
Gravity particles	Based on gameplay physics update

Visible stepping must not occur.

18.3 Quality Tiers

High

- Full sky gradient.
- Dynamic celestial glow.
- More stars.
- Dynamic shadow direction.
- Water reflections.
- Additional environmental particles.
- Full ambient transitions.

Medium

- Full sky gradient.
- Sun and moon movement.
- Reduced stars.
- Simplified shadows.
- Reduced particles.
- Simplified reflections.

Low

- Gradient or color-ramp sky.
- Sun and moon movement.
- Minimal stars.
- Static or disabled dynamic shadows.
- Minimal environmental particles.
- No expensive reflections.
- Readability lighting preserved.

Performance settings must not change competitive gameplay.

19. Accessibility

19.1 Reduced Motion

A reduced-motion option may:

- Slow or simplify cloud movement.
- Remove camera impulses.
- Reduce environmental particles.
- Remove rapid gravity streaks.
- Reduce background animal movement.
- Simplify celestial parallax.

The day/night colors may continue changing.

19.2 Reduced Visual Effects

A reduced-effects option may:

- Disable nonessential glow.
- Reduce stars.
- Disable water reflections.
- Reduce fireflies.
- Reduce atmospheric fog.
- Simplify shadow animation.

19.3 Night Visibility

A night-visibility option may increase:

- Player outline.
- Pickup outline.
- Rim-light strength.
- Background separation.

This must not reveal hidden information or create a competitive advantage beyond reasonable accessibility support.

The game may standardize the setting in ranked play if testing determines that unrestricted values create material advantages.

19.4 Color Vision

Important gameplay objects cannot rely only on sky-relative color.

Green Apples, Purple Fruits, Golden Hero Tokens, hazards, and player indicators should use:

- Shape.
- Iconography.
- Outline.
- Animation.
- Contrast.
- Color.

Sunset or nighttime palettes must be tested against common color-vision deficiencies.

19.5 Photosensitivity

Avoid:

- Rapid celestial flashes.
 - Lightning tied to normal cycle transitions.
 - Sudden full-screen brightness changes.
 - Star flickering.
 - Repeated high-contrast pulses.
 - Abrupt exposure shifts.
-

20. Development Phases

Phase 0: Existing-System Audit and Decoupling

Objective

Identify every location where environment darkness is currently connected to gravity and separate those responsibilities.

Required Work

- Locate gravity-to-darkness calculations.
- Locate shaders receiving gravity-derived light values.
- Locate global variables combining match difficulty and environment phase.
- Document the current gravity curve.
- Document the current lighting implementation.
- Ensure removing the relationship does not alter gameplay gravity.
- Create separate environment-time and gameplay-gravity values.
- Add debug readouts for both systems.

Debug Display

During development, expose:

```
Match elapsed time  
Cycle phase  
Cycle duration  
Starting phase  
Effective gravity  
Base gravity  
Gravity ramp
```

Purple Fruit modifier
Current environment phase name
Environment synchronization drift

Completion Criteria

- Gravity can increase while the environment remains fixed.
- The environment can cycle while gravity remains fixed.
- Purple Fruit can affect one player without changing the global sky.
- No gameplay calculation reads sky darkness.
- No environment calculation reads effective gravity.

Phase 1: Core Day/Night MVP

Objective

Create the complete 120-second loop using minimal visual elements.

Required Work

- Add normalized cycle timer.
- Add fixed sunrise starting phase.
- Add sky color gradient.
- Add sun movement.
- Add moon movement.
- Add star opacity.
- Add seamless wrap from 120 seconds to zero.
- Keep UI unaffected.
- Use one test biome.

Excluded From This Phase

- Dynamic shadows.
- Background animals.
- Advanced ambience.
- Random starting phases.
- Network synchronization.
- Complex water reflections.
- Seasonal events.

Completion Criteria

- A full cycle completes in exactly 120 seconds.

- The loop contains no visible reset.
 - Sun and moon arcs are stable in portrait mode.
 - Gravity can be independently adjusted.
 - The environment remains readable during night.
 - The feature runs reliably on the target Android device.
-

Phase 2: Competitive Readability and Art Polish

Objective

Make every phase visually attractive while protecting gameplay clarity.

Required Work

- Author sunrise, day, sunset, twilight, night, and predawn palettes.
- Add player rim lighting.
- Add pickup readability protection.
- Validate Golden Hero Tokens against sunset.
- Validate Purple Fruits against twilight and night.
- Validate Green Apples against daytime foliage.
- Tune star density.
- Tune celestial brightness.
- Add controlled cloud brightness.
- Add shadow-opacity transitions.
- Add editor neutral-lighting mode.
- Add Preview Game Lighting mode.

Completion Criteria

- Gameplay objects remain identifiable in every phase.
 - No phase provides a meaningful visibility advantage.
 - Night is atmospheric but not excessively dark.
 - Sunset does not obscure gold, orange, yellow, or red objects.
 - Effects remain readable without washing out the scene.
-

Phase 3: Online Synchronization

Objective

Ensure both players see the same cycle state.

Required Work

- Add environment seed to match configuration.
- Add authoritative starting phase.
- Add authoritative match start time.
- Derive phase from synchronized elapsed time.
- Add drift detection.
- Add drift correction.
- Validate reconnect behavior.
- Validate bot matches.
- Validate private head-to-head matches.
- Validate replays and spectators where available.

Completion Criteria

- Two devices remain visually synchronized throughout a match.
 - Repeated tests produce the same environment from the same seed.
 - Network lag does not permanently desynchronize the cycle.
 - Environmental synchronization does not increase gameplay latency.
 - No per-frame environment state is required from the server.
-

Phase 4: Starting-Phase Variety

Objective

Prevent most matches from repeatedly showing the same portion of the cycle.

Required Work

- Add curated phase anchors.
- Select starting anchor from match seed.
- Ensure both players receive the same anchor.
- Add analytics for phase distribution.
- Confirm each anchor preserves readability.
- Confirm transitions remain seamless from every starting point.

Completion Criteria

- Players encounter a reasonable distribution of starting phases.
 - No starting phase appears disproportionately.
 - All starting phases are competitively neutral.
 - A complete two-minute survival still completes one full cycle.
-

Phase 5: Biome Reactions and Environmental Audio

Objective

Make the environment feel alive without distracting from competitive play.

Required Work

- Add day-active and night-active cosmetic reactions.
- Add artificial-light activation.
- Add water reflection changes where supported.
- Add ambient audio crossfades.
- Add deterministic background-event scheduling.
- Create biome profiles.
- Establish competitive effect-density limits.
- Confirm background reactions do not resemble pickups or hazards.

Completion Criteria

- Each biome has a distinct day and night identity.
 - Environmental reactions remain peripheral.
 - Ambient audio never masks gameplay cues.
 - Cosmetic events reproduce correctly in replays where required.
 - Low-performance devices can disable expensive reactions.
-

Phase 6: Gravity Presentation Refinement

Objective

Replace the former gravity-darkness relationship with intuitive physical feedback.

Required Work

- Add gravity strength normalization.
- Add downward particle behavior.
- Add character squash and motion response.
- Add gravity threshold feedback.
- Add Purple Fruit-specific presentation.
- Add gravity audio layer.
- Add accessibility controls.
- Tune feedback so it is noticeable without becoming overwhelming.

Completion Criteria

- Players can feel that gravity is increasing without relying only on a number.
 - Gravity presentation remains understandable at every time of day.
 - Gravity effects do not obscure the player.
 - Purple Fruit is clearly distinguishable from normal gravity escalation.
 - One player's gravity debuff never changes the opponent's environment.
-

Phase 7: Optimization, QA, and Release

Objective

Prepare the feature for production.

Required Work

- Profile GPU cost.
- Profile shader cost.
- Profile transparency overdraw.
- Test multiple Android quality tiers.
- Validate battery impact.
- Validate thermal behavior.
- Validate 30 FPS and 60 FPS timing.
- Test app interruptions.
- Test network reconnection.
- Test full two-minute loops repeatedly.
- Test cycle wrap.
- Test every biome.
- Test every competitive pickup.
- Test representative skins and particle effects.
- Test accessibility settings.
- Test spectator and replay behavior.
- Confirm analytics events.

Completion Criteria

- No gameplay regression.
- No material competitive visibility difference.
- No cycle drift between devices.
- No visible loop discontinuity.
- No significant frame-rate instability.
- No shader compilation hitch during a live match.
- No asset-loading hitch at phase transitions.
- No pay-to-win or cosmetic readability violation.

21. QA Test Matrix

The following cases should be explicitly tested.

Timing

- Cycle begins at phase zero.
- Cycle begins at every curated anchor.
- Cycle reaches the expected phase at 30 seconds.
- Cycle reaches the expected phase at 60 seconds.
- Cycle reaches the expected phase at 90 seconds.
- Cycle wraps at 120 seconds.
- Cycle remains stable after multiple loops in a test environment.
- Cycle behaves correctly at 30 FPS.
- Cycle behaves correctly at 60 FPS.
- Cycle behaves correctly under frame drops.

Gravity Independence

- Base gravity changed while cycle remains unchanged.
- Gravity ramp disabled while cycle continues.
- Cycle frozen while gravity continues in debug mode.
- Purple Fruit collected by Player A.
- Purple Fruit collected by Player B.
- Both players collect Purple Fruit.
- Gravity reaches extreme values during day.
- Gravity reaches extreme values during night.

Competitive Synchronization

- Two physical devices.
- Different device frame rates.
- Different graphics-quality settings.
- Temporary network latency spike.
- Temporary packet loss.
- App background and return where supported.
- Spectator joins after match start.
- Replay starts from zero.
- Replay scrubs to a later point where supported.
- Bot backfill match.
- Ghost opponent match.
- Private head-to-head match.
- Ranked match.

Readability

Test each critical object during:

- Sunrise.
- Day.
- Sunset.
- Twilight.
- Night.
- Predawn.

Objects:

- Default character.
- Bright character skin.
- Dark character skin.
- Purple character skin.
- Green Apple.
- Purple Fruit.
- Golden Hero Token.
- Hazards.
- Tap effects.
- Character particles.
- Opponent outline.
- Match timer.
- Gravity meter.
- Network warning.

Performance

- Entry-level supported Android device.
- Midrange Android device.
- High-end Android device.
- Battery-saver mode.
- Thermal throttling.
- Maximum particle load.
- Two players with complex cosmetic effects.
- Night with maximum stars and lights.
- Sunset with gradient and clouds.
- Repeated match restarts.

22. Analytics

The feature should provide enough information to identify performance or readability problems without collecting unnecessary data.

Recommended events and properties:

Match Environment Initialized

```
match_id
mode
biome_id
profile_id
starting_phase
environment_seed
graphics_quality
device_performance_class
```

Match Ended

```
survival_duration
ending_phase
maximum_gravity
starting_phase
graphics_quality
average_fps
minimum_fps
```

Environment Resynchronized

```
drift_amount
correction_method
match_elapsed_time
device_type
```

Accessibility Setting Used

```
reduced_motion
reduced_effects
night_visibility
outline_strength
```

Analytics should help answer:

- Do specific starting phases correlate with unusual loss rates?
- Do players leave matches beginning at night more often?

- Are low-end devices dropping frames during particular phases?
- Does sunset reduce pickup collection rates?
- Are synchronization corrections happening too frequently?
- Do accessibility settings materially alter competitive outcomes?

A correlation does not automatically mean causation, but unexpected differences should trigger review.

23. Acceptance Criteria

The feature is considered ready for production when all of the following are true:

1. The competitive environment completes a seamless cycle every 120 seconds.
2. The sun and moon visibly move through the sky.
3. Gravity progression and environment progression are fully decoupled.
4. Purple Fruit affects gameplay gravity without altering global time of day.
5. Both players receive the same environment phase in synchronized matches.
6. The environment remains synchronized without per-frame server updates.
7. Nighttime does not obscure players, pickups, hazards, or interface elements.
8. Golden Hero Tokens remain readable during sunset.
9. Green Apples remain readable against green biomes.
10. Purple Fruits remain readable during twilight and night.
11. The feature maintains target performance across supported Android hardware.
12. Lower graphics settings preserve the cycle's identity.
13. No cosmetic purchase changes competitive visibility or environmental advantage.
14. The full cycle can be reproduced from saved match data.
15. The editor provides a neutral-lighting workflow.
16. Gravity is communicated through physical feedback rather than darkness.
17. Cycle transitions do not interrupt music or trigger asset-loading hitches.

18. Background reactions remain cosmetic and non-distracting.

24. Explicit Guardrails

The developer should not:

- Make the sky darker because gravity increases.
 - Make gravity stronger because it becomes night.
 - Give nighttime different item probabilities.
 - Change tap force at sunrise or sunset.
 - Reduce opponent visibility at night.
 - Allow paid skins to glow more clearly than ordinary characters in ranked play.
 - Use large lens flares over the gameplay region.
 - Flash the screen during sunrise or sunset.
 - Restart music at each cycle phase.
 - Load major assets during a live phase transition.
 - Allow clients to independently select competitive starting phases.
 - Make decorative animals interfere with collision.
 - Let background effects resemble collectible items.
 - Use weather or celestial events that hide gameplay information.
 - Tie Purple Fruit effects to the global environment.
 - use the environment controller as the authoritative match timer.
 - make the day/night system necessary for bot decision-making.
-

25. Future Expansion Opportunities

The architecture should leave room for future cosmetic additions without placing them into the initial scope.

Potential future systems include:

- Seasonal sun and moon variants.
- Different moon phases.
- Aurora events.
- Meteor showers.
- Rare comets.
- Holiday skies.
- Biome-specific celestial bodies.
- Distant storms.
- Eclipse events.
- Player-selectable private-match atmosphere.
- Tournament-specific skies.
- Ranked-season environmental themes.

- Victory screens based on the ending phase.
- Profile statistics for surviving a complete cycle.
- Achievements for surviving from sunrise to sunrise.
- Secret cosmetic unlocks associated with long survival.
- Special spectator presentation.
- Replay cinematic controls.

Any future competitive celestial event must remain readable, synchronized, cosmetic, and non-pay-to-win.

26. Final Design Principle

This feature is successful when the player feels two forms of progression occurring simultaneously:

- The match becomes mechanically harder because gravity is increasing.
- The world visibly moves through time because the day/night cycle is advancing.

The player should understand these systems intuitively without confusing them.

The world becomes darker because night has arrived.

The character falls faster because gravity has become stronger.

Those events may occur near one another during a match, but they are not the same event and must never be implemented as though one causes the other.

Time changes the atmosphere. Gravity changes movement. Skill determines who stays airborne.